

SwimForce

Cole Griscavage & Omar Ebied advised by Matt Zucker

Swarthmore College, Department of Engineering

April 21, 2024

Abstract

The SwimForce system aims to provide a swimmer tracking system developed to address limitations in current swimming analysis tools. The core design challenge was to develop an automated, swimmer-specific tracking mechanism capable of detecting individual athletes in real time and collecting video footage of their swim turns to further analyze. SwimForce integrates ultrasonic transducers for swimmer identification, hydrophones for signal detection, a digital acquisition system for real-time frequency analysis, and underwater cameras for personalized recordings. We describe the process of selecting components for underwater durability and signal clarity, designing signal-processing circuits, implementing FFT-based frequency detection, and synchronizing video recording. The system was tested in controlled aquatic environments and evaluated based on its ability to correctly identify swimmer entries and sort their video clips.

Keywords: Ultrasonic Transducers, OpenCV, Hydrophone, Analog Discovery 2, Underwater System

1 Introduction

We proposed a comprehensive swimmer tracking system designed to improve performance analysis in swimming. The core objective of SwimForce is to provide precise and detailed data on a swimmer's performance, thereby enhancing the effectiveness of training and coaching. This is achieved through the integration of several key technological components, each playing a crucial role in the system's functionality.

1.1 Motivation

Swimmer tracking presents unique challenges that conventional video recording systems struggle to address effectively. While the aquatic environment already poses obstacles ranging from water damaged hardware to signal distortion, it's difficult to capture reliable data and implement real-time tracking. Additionally, with frequent changes in stroke and underwater movements, a system that can maintain continuous identification of specific athletes in an environment with multiple swimmers is necessary. This is especially critical during team practices when multiple swimmers occupy the same lane. Traditional analysis methods that rely solely on visual identification often fail to distinguish between swimmers with similar appearances. These technical constraints have persistently limited the capabilities of swimmer tracking systems, thus creating a gap between the analytical needs of coaches and athletes as well as the tools readily available to them.

1.2 Limitations of Existing Systems

Current commercial systems like Poolside Scout offer basic video recoding capabilities that fall short in providing the necessary features for comprehensive performance analysis. While Poolside Scout delivers accessible video footage that can be recorded and slowed down, it primarily serves as a passive recording tool without automated swimmer identification or real-time data processing capabilities. Most importantly, these systems rarely provide automated organization of footage by swimmer. Thus forcing coaches to manually analyze footage, resulting in extensive post-recording sorting and analysis that diverts valuable coaching time from direct athlete interaction. In turn, this time-consuming process reduces the efficiency of training sessions.

1.3 Background

The development of the SwimForce system emerges from the intersection of competitive swimming experience and technical expertise in both hardware and software domains. This interdisciplinary approach informs the system design, which focuses primarily on acquiring and analyzing performance data in real-time conditions. The experience of current training methodologies in elite swimming environments reveals several persistent limitations that divert from effective training sessions.

Currently available systems, such as Poolside Scout, offer basic video recording with limited analytical capabilities, primarily focused on manual replay and clip segmentation. However, these tools exhibit several significant technical limitations with no automated identification, continuous manual oversight for video capture, and no real-time data linkage between swimmer identity and the recorded footage. Additionally, their subscription-based pricing model poses resource constraints for smaller teams and personal training.

The technical approach employed in this research utilizes ultrasonic transducers operating at frequencies between 20-100 kHz for swimmer identification, paired with hydrophones for signal detection and underwater cameras for motion capture. Signal processing is handled through digital acquisition systems capable of real-time frequency analysis. This integrated system architecture represents an attempt to address the technical limitations identified in current training environments.

2 Engineering Design

The primary goal of the SwimForce system is to enable swimmer-specific video capture using real-time audio signaling. This requires the system to detect uniquely emitted ultrasonic signals from a wearable device, identify the presence of a swimmer near the cameras field of view, and automatically trigger and store synchronized underwater recording. Key success criteria include reliable signal detection within a pool environment, robust hardware waterproofing, system responsiveness under 1 second, and successful storage of clipped video segments labeled by their corresponding swimmer.

2.1 Professional Standards and Codes

SwimForce relies on adherence to the IP68 standard for waterproofing to ensure safe and reliable operation of all submerged electronic components. IP68 specifies that enclosures must protect

against continuous immersion in water beyond 1 meter and offer complete protection against dust ingress. All swimmer-worn devices, including transducers and power modules, were designed or selected with IP68 compliance as a core constraint. This standard informs both component selection and enclosure design, particularly the use of sealed IP68 rated plastic housings, and epoxy sealing for holes to reveal transducer emitter. Meeting IP68 standards is essential for ensuring long-term durability and remaining operational in a chlorinated pool environment.

2.2 Components

2.2.1 *Ultrasonic Transducers*

A cornerstone of SwimForce is the use of ultrasonic transducers, which emit high-frequency acoustic waves, typically in the 20 kHz to 100 kHz range. They convert electrical energy into high-frequency sound waves (ultrasound) and vice versa, allowing them to be versatile in a variety of applications, including medical imaging, industrial inspection, and sonar. These transducers typically rely on piezoelectric materials, which generate mechanical vibrations when an electrical voltage is applied. Consequently, their emitted frequency can be adjusted, permitting the system to assign different frequencies to different swimmers for multi-swimmer tracking. Multiple different types of transducers were tested, but the larger CUSA-TR801-065-2000-TH68, and smaller CUSA-TR80-065-2000-TH68 were both thoroughly tested. Both of these transducers are IP68 rated, meaning they are completely water proof at the depths required within the pool. The larger transducer is 18 mm in diameter while the smaller transducer is 16 mm in diameter. The smaller transducer had an additional lip that was 12 mm in diameter. These transducers were optimized for maximum output at 40 KHz, and had maximum operating voltages of 150 V (Same Sky, 2024). After testing, it appeared that the smaller transducer generated a louder signal at 3V, 5V, 9V and 12V. Although these transducers could be used to receive signals as well, the hydrophone yielded better results. The smaller CUSA-TR80-065-2000-TH68 transducer was used in the final design. A smaller transducer was tested as part of a prototype for a smaller design that was never used in the pool. The CUSA-TR65-065-2200-W68 is a 14 mm transducer that peaks at 40 KHz and operates up to 150V (Same Sky). This transducer was able to produce a signal of similar amplitude, but was not used in the final design.

2.2.2 *Underwater Cameras*

SwimForce uses a singular submersible POE (Power Over Ethernet) camera setup to provide detailed visual data. These devices offer high-resolution video feeds while minimizing cabling complexity, as both power and data are transmitted through a single Cat5e/6 cable. The specific cameras used are stainless steel underwater POE network cameras with an HD 1080p feed running at 30 fps through a 4 mm fixed lens. It uses an RTSP (Real-Time Streaming Protocol) in order to stream its footage in real time over IP networks allowing a seamless wireless connection (Sideline Scout).

2.2.3 *Hydrophone*

The hydrophone used in SwimForce functions as a broadband acoustic sensor, designed to pick up ultrasonic pulses in water. These sensors operate based on the same piezoelectric principles

as the transducers, but are optimized for receiving signals underwater. Typical hydrophones used in controlled aquatic environments have a frequency range of 10 kHz to 100 kHz and a receiving sensitivity near -200 dB re 1V/ μ Pa. It is essentially an underwater microphone; A 8103 type hydrophone manufactured to be omni-directional and read signals up to 180 KHz (Hottinger Brüel & Kjær A/S., 2021).

2.2.4 *Analog Discovery 2*

To interface with the analog signals from the hydrophone and ultrasonic transducers, SwimForce employs the Analog Discovery 2 by Digilent, an all-in-one USB oscilloscope, logic analyzer, and waveform generator. This compact device provides two analog input channels with 14-bit resolution and a sampling rate of up to 100 MS/s (Digilent). This enables precise digitization of high-frequency acoustic signals in real time. The device connects to a PC via USB and is controlled through Digilent's WaveForms software, which supports automated data acquisition via a scriptable API. Its portability, signal accuracy, and multi-functionality make it well suited for the experimental and embedded signal processing for SwimForce. It could be used to measure the intensity of different analog signals that the hydrophone received as it can read a maximum frequency of 9 MHz, which is far beyond the 40 KHz required for this scope (Digilent). While the Analog Discovery 2 is ideal for rapid prototyping and development due to its versatility and ease of use, a fully deployable version of SwimForce would require a more specialized, embedded acquisition system optimized for real-time processing, power efficiency, and waterproof integration.

2.2.5 *Arduino Pro Mini*

The Arduino Pro Mini is one of the smallest Arduino boards, measuring 18 mm by 33 mm. The board takes a raw input from 3.3V to 12V, and runs on 3.3V. The board comes with a ATmega328P processor with a clock speed of 8 MHz. This board was chosen for its processing capabilities, low operational voltage, and small size. The Pro Mini has 6 PWM pins, which were programmed to generate the 40 KHz signal. The clock speed of 8 MHz provided headroom for generating a 40 KHz signal (Arduino).

2.2.6 *Amplifier*

The Brüel & Kjær 2635 amplifier is a four-stage amplifier consisting of an input amplifier, a low-pass filter amplifier, an integrator amplifier, an integrator amplifier, and an output amplifier. This amp has a built-in battery-operated power supply (Brüel & Kjær, 1989).

2.2.7 *Tip 122*

The Tip 122 is a transistor that is used in this context to regulate the flow of power into the transducers. The Tip 122 has three connections: the base, collector and emitters. The collector is like the inlets of power, while the emitter is the outlet of the power. Power is only able to flow through from the collector to the emitter when current is applied to the base (OnSemi, 2017).

2.2.8 Batteries

Two types of batteries were used for testing. First was a typical 9V alkaline battery. These batteries are not rechargeable, are relative large in size (48.5 mm x 26.5 mm x 17.5 mm), and have a short life span. During test, these batteries drained quickly and needed to be replaced a couple of times. The 9V battery was replaced with a smaller 3.7V lithium-ion battery. The lithium batteries are 110 mAh, are rechargeable, and 25 mm x 15 mm x 4 mm. The lithium batteries easily fit into the larger case, but were small enough to fit into the smaller design. The lithium batteries could charge in about 1 hour and provided a maximum discharge rate of 22 mA, which turned out to be plenty for this application (DTP).

2.2.9 Boost Converter

As mentioned in the previous section, the original circuit was powered by a 9V battery. This battery was connected directly to the transducers, meaning the transducers were powered by 9V. Through testing, it became clear that increasing the voltage to the transducers would increase the amplitude of their signal output. When designing the smaller device, it was necessary to switch to a lithium-ion battery, due to their compact size and relative energy density. These batteries produce a voltage of 3.7V, which is far too low to produce a significant signal from the transducers. A 5644 boost converter, manufactured by Adafruit industries was implemented in the circuit to step the voltage up from the battery. The 5644 takes an input voltage between 1.8V and 6V and steps it up to 12V. The 5644 can handle up to 400 mA, which is far more wattage than the battery can output in the first place, meaning the it wouldn't be a limiting factor (Adafruit Industries).

2.2.10 Waterproof Encasing

An IP68 Junction Box was used to create the case for all of the electronics in the beeper. The box is 2.48" x 2.28" x 1.38" and is seal by 4 screws at each corner. The box is made of Acrylonitrile butadiene styrene (Raculety, 2020). Two 12 mm holes were drilled in the sides of the box to create space for the transducers. The transducers were fitted into these holes and then glued into place and water proofed with epoxy.

2.2.11 OpenCV

OpenCV (Open Source Computer Vision Library) was used to explore video segmentation and motion detection. While full swimmer velocity tracking was not implemented, OpenCV enabled initial tests with background subtraction and object detection. Its real-time processing capability, along with support for Python, makes it a valuable tool for future iterations of SwimForce that may include optical flow-based velocity estimation.

Through the integration of these technologies, each carefully selected for its technical performance and reliability, SwimForce delivers precise, real-time swimmer analytics that support elite training and research in aquatic performance.

2.3 Requirements

SwimForce was developed to meet a defined set of functional and technical requirements, each designed to be verifiable through practical testing. A core requirement is the ability to emit ultrasonic signals at adjustable frequencies through custom-built transducers, enabling the unique identification of swimmers. The hydrophone system must reliably detect these signals and initiate recording only when a swimmer is present within a specified range (Approximately 2 meters from the wall). Verification was performed by recording signal amplitude using FFT analysis and confirming detection accuracy under controlled test conditions. The integration of POE cameras is essential to provide high-resolution videos from underwater, with real-time synchronization between video and audio data streams where each clip must be cut to approximately 30 seconds and tagged with metadata linking it to the detected swimmer ID. These requirements ensure that SwimForce not only captures precise technical data but also facilitates intense training. System testing can confirm that the video clips consistently started within 0.5 seconds of the signal detection and were correctly labeled. By simulating multiple swimmer passes and confirming appropriate detection thresholds, clip durations, and tagging accuracy, all requirements can be met through rigorous testing.

2.4 Constraints

The development of SwimForce is guided by several critical constraints, primarily stemming from its need for underwater functionality and demand for real time data collection. All hardware components, including cameras, transducers, and hydrophone, must comply with IP68 waterproofing standards and built from corrosion-resistant materials capable of long term exposure to chlorinated water. Considering the hydrophone and underwater cameras are manufactured to completely withstand these conditions, the waterproofing solely pertains to the ultrasonic transducer. Regardless, these constraints can be met by fully submerging each enclosure in the pool for extended periods of time and proving full functionality throughout. Additionally, The system's ultrasonic identification must remain operable for long periods of time, requiring careful selection of ultrasonic frequencies that maximize underwater signal clarity and a rechargeable power source. Wearable transducers must operate for at least 30 minutes using compact lithium-ion batteries which can be verified by measuring signal amplitude at various discharge levels and confirming reliable detection throughout the battery cycle. Finally, constraints on system data load required that video clips remain short and storage-efficient. The clip segmentation process was tested to ensure that recording durations did not exceed 10 seconds per trigger event, thereby preventing excessive accumulation of unused footage. These constraints framed the SwimForce design and informed decisions on materials, signal frequency, enclosure engineering, and data handling.

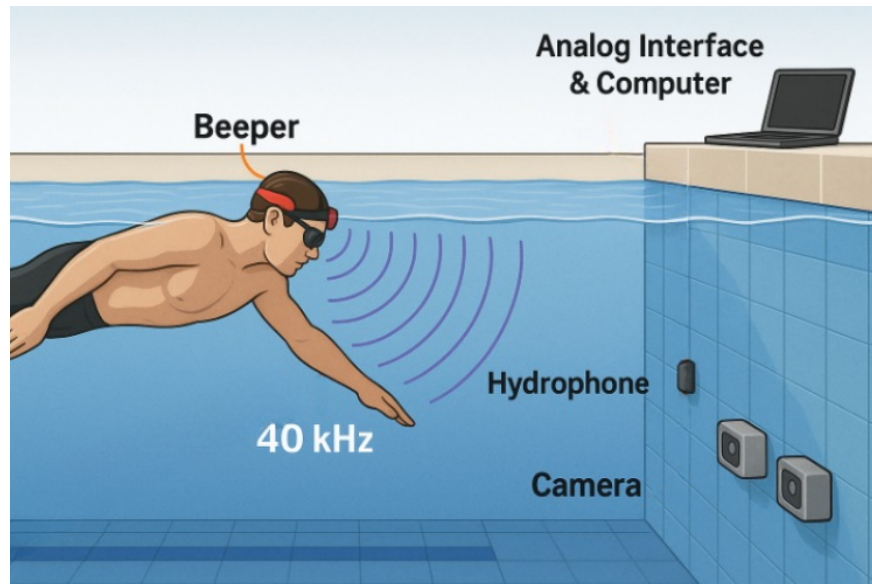


Figure 1: The wearable ultrasonic “beeper” emits 40 kHz signals, detected by a hydrophone near the pool wall. The signal is digitized by an analog interface and triggers synchronized video capture from underwater cameras

3 Technical Discussion

Ultrasonic communication plays a central role in SwimForce’s swimmer identification system. In an aquatic environment, sound travels significantly faster and more reliably than in air, approximately 1500 meters per second in water compared to 350 meters per second in air (NOAA Ocean Exploration, 2002). However, the propagation of ultrasonic signals underwater is influenced by several factors including temperature, salinity, and water pressure, all of which affect the speed and attenuation of sound waves. Frequencies in the range of 20 kHz to 100 kHz are commonly used for underwater detection systems, as they provide a balance between signal range and resolution. In SwimForce, transducers emit customized ultrasonic signals that are detected by hydrophones, allowing swimmer presence to be inferred and camera systems to be triggered for data capture.

Before designing the final circuit for the transducer, it was necessary to define clear functional requirements: the circuit needed to generate a stable ultrasonic signal at a target frequency (approximately 40 kHz), deliver adequate power to the transducer for loud and detectable outputs, and remain compact for hydrodynamic swimmer use. Additionally, the design had to operate efficiently from a low-voltage lithium-ion batteries while providing a reliable method to turn on and off the system.

For visual data acquisition, SwimForce uses a swiveled underwater POE camera mounted on the wall of the last lane. While OpenCV was explored for swimmer tracking and velocity estimation, it was not implemented in the final system due to the complexities of velocity calculating with such narrow video data. Instead, the camera’s orientation and angle were adjusted to minimize issues caused by underwater light distortion, refraction, and particulates. This allowed for clear footage of the swimmer’s flip turns, which was segmented automatically in response to ultrasonic signal detection.

SwimForce also integrates principles from digital and embedded systems to synchronize its

hardware components. The system uses the Analog Discovery 2 in order to continuously sample hydrophone data and monitor for specific frequency thresholds. Upon detection, the program triggers the POE camera system to begin video capture. This synchronization is critical, as the swimmer's movement must be recorded precisely at the moment of audio detection to ensure accurate swimmer capture. Data streams from both the ultrasonic and visual components are timestamped and merged in real-time, facilitated by embedded software written in Python.

3.1 Circuit Analysis

The ultrasonic transducer circuit is designed to emit pulses at adjustable frequencies using a square wave generator and an amplification stage. The Arduino acts as a waveform generator, which creates a high-frequency square wave signal from its PWM output. This signal is then fed into a TIP-122 transistor, which serves as a switch to modulate a 12V power supply to the transducers. The transducers are connected in parallel to each other. The transistor's base is connected through a 1k Ohm resistor. The 12V power supply is connected to a 3.7V lithium ion battery. The Arduino board is connected directly to the 3.7V lithium ion battery and in parallel with the 12V power supply. The 12V power supply is a boost converter, which takes any power input between 1.8V and 6V, and steps it up to 12V.

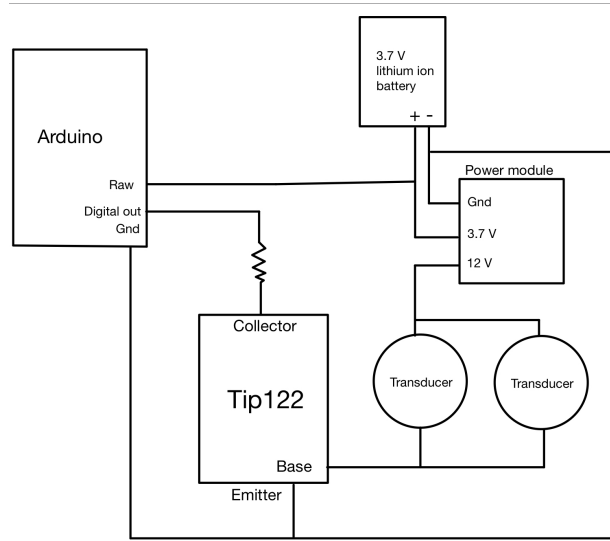


Figure 2: Ultrasonic circuit inside the beeper

3.2 Brüel & Kjær Amplifier

The Brüel & Kjær 2635 Accelerometer Amplifier was used to boost the signal from the hydrophone. This amplifier has many different settings and had to be carefully tuned to boost the signals from the beeper without generating a lot of noise. First, the amplifier had to be set to a transducer sensitivity of $.102 \text{ pC/m/s}^2$, which was the sensitivity listed for the hydrophone. Next, the low pass filter needed to be adjusted to the $>100 \text{ kHz}$ setting, which allowed for signals in the 40kHz range. The amplifier has multiple different amplification modes. The acceleration mode

proved to be the best for this application because it still maintains adequate amplitude in the ultrasonic range, while the other amplification modes dropped off in the upper frequencies. Multiple different levels of amplification were tested, but a level of 100 mV/Unit Out provided the cleanest signal, by creating adequate amplitude of the 40kHz signal without generating excess noise.

3.3 Arduino Pro Mini Code

The wearable device responsible for emitting ultrasonic signals to uniquely identifying the swimmer is built around the Arduino Pro Mini. The compact and low-power micro controller allows for the ultrasonic transducer to beep at 40 kHz using hardware based PWM as shown in Appendix A.

The code leverage Timer2 on the ATmega328P micro controller to generate a square wave with frequency of 40 kHz on digital pin 11. Timer 2 is configured in fast PWM mode by manipulating control registers directly. This proves a better frequency accuracy and timing resolution that the standard and more commonly used ‘analogWrite()’ function

```

1 void setup() {
2     pinMode(11, OUTPUT);
3     TCCR2A = _BV(COM2A0) | _BV(COM2B1) | _BV(WGM20);
4     TCCR2B = _BV(WGM22) | _BV(CS20);
5     OCR2A = 52;
6     OCR2B = 50;
7 }

```

The values of ‘OCR2A = 52’ and ‘OCR2B = 50’ set the output frequency close to 40 kHz (approximately 38 kHz) by adjusting the timer’s output compare values. The transducer is toggled on for 200 milliseconds and then turned off for 800 milliseconds by setting pin 11 to input mode. This duty cycle ensures that each swimmer emits a periodic and distinguishable ultrasonic signal, while conserving battery life between each pulses.

3.4 Fast Fourier Transform (FFT)

To extract meaningful information from the hydrophone signal, the system performs a Fast Fourier Transform (FFT) using NumPy and other Python based libraries. FFT enables the conversion of time-domain auditory signals into their frequency-domain representation. This allows the system to detect the presence of specific ultrasonic frequencies corresponding to swimmer IDs. The FFT is implemented on hydrophone data acquired through the Analog Discovery 2 interface using the WaveForms SDK. While Digilent’s WaveForms application provides a user-friendly spectrum analyzer for the Analog Discovery 2, it could not be directly integrated into the autonomous system. As a result, this spectrum analysis replicates the functionality of the WaveForms software interface, but with additional flexibility and integration of real-time detection. Peaks in the FFT spectrum are matched to predefined frequencies, where each swimmer has a different frequency range in order to confirm presence and trigger the video capture process.

Hydrophone signals are sampled at 250 kHz with a buffer size of 16,384 samples. These parameters match the WaveForms app settings and provide sufficient resolution near the 40 kHz swimmer tag frequency. To isolate frequency content in the swimmer’s expected range, a 4th-order Butterworth bandpass filter was applied. This eliminates low-frequency noise and irrelevant high-

frequency components. The filter was applied to the raw signal data for each swimmer's designated frequency band before FFT processing.:

```

1 def butter_bandpass(lowcut, highcut, fs, order=5):
2     nyq = 0.5 * fs
3     low = lowcut / nyq
4     high = highcut / nyq
5     b, a = butter(order, [low, high], btype='band')
6     return b, a
7
8 def butter_bandpass_filter(data, lowcut, highcut, fs, order=5):
9     b, a = butter_bandpass(lowcut, highcut, fs, order=order)
10    y = lfilter(b, a, data)
11    return y

```

To reduce spectral leakage from edge cases in the acquired signal, we applied a blackman window to the filtered data:

```

1 window = np.blackman(len(filtered_data))
2 windowed = filtered_data * window
3 correction_factor = np.sum(window) / len(filtered_data)

```

Now, the FFT calculation can be done using NumPy's FFT routine in order to convert the resulting magnitudes to decibel volts (dBV).

```

1 fft_result = np.fft.rfft(windowed) / (BUFFER_SIZE * correction_factor)
2 magnitudes = 20 * np.log10(np.abs(fft_result) + 1e-12)

```

Finally, the system identifies the frequency range associated with each swimmer and checks whether the peak dBV exceeds a predefined threshold. If the condition is met, the POE cameras trigger and a 10 second long clip is saved with the corresponding swimmers name as the title.

This processing pipeline replicates the functionality of WaveForms in a programmable environment and enables reliable swimmer detection in real time. A full version of this detection loop is provided in Appendix C.

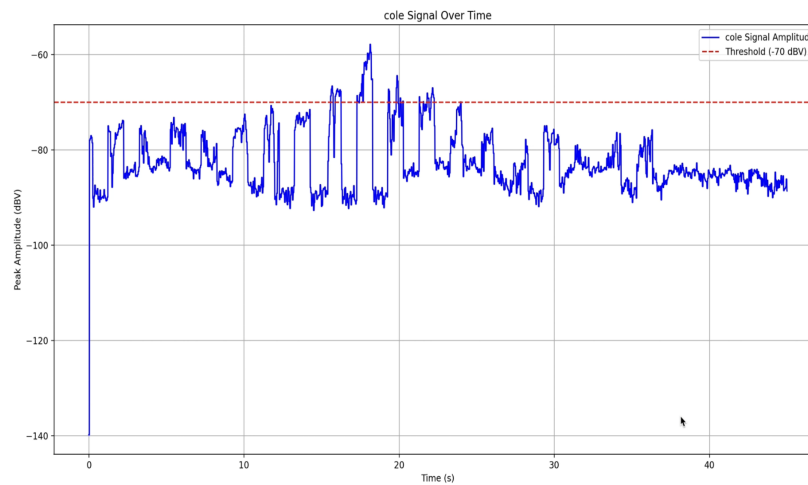


Figure 3: Sample time-series plot showing the peak FFT amplitude (in dBV) of the filtered hydrophone signal across a 45-second interval. The red dashed line indicates the detection threshold

3.5 Camera Clipping: ‘ffmpeg’

To capture the underwater video footage, the open-source multimedia framework `ffmpeg` is used. `ffmpeg` is a powerful command-line tool used for processing, streaming, and converting video data as it supports a wide range of codecs, protocols, and input/output formats.

The underwater POE cameras stream video using the Real-Time Streaming Protocol (RTSP). RTSP is a network control protocol designed for real time media streaming. It enables low latency video transmission over IP networks and allows clients to start, stop, and control video feeds on demand. In SwimForce, the cameras RTSP stream is accessed via a dedicated IP address and port using TCP transport to ensure stream stability (Sideline Scout).

When the system detects a swimmer’s ultrasonic signal using the FFT computation, the function `rec_clip()` is called to constructs an `ffmpeg` command that connects to the camera via RTSP using TCP, records for the specified 10 second duration, encodes the output with the H.264 codex using the `libx264` encoder, and save the result as an MP4 file with the swimmers name as the filename.

```

1 command = [
2     "ffmpeg",
3     "-rtsp_transport", "tcp",
4     "-y",
5     "-t", str(durr),
6     "-i", RTSP0, # First input stream
7     "-c:v", "libx264",
8     "-preset", "ultrafast",
9     "-crf", "23",
10    "-pix_fmt", "yuv420p",
11    output_filename
12 ]

```

The use of `ffmpeg`’s `-preset ultrafast` and `-crf 23` options balances recording speed and video quality. This approach enables low-latency capture without dropping any frames (SAFR Support, 2021). The full implementation of this camera clipping function is provided in Appendix B.

4 Implementation

The set up at the pool can be broken into three separate systems that all interact with each other wirelessly. First, is the beeper, which is a battery operated embedded system that goes into the pool and can move freely in the water. The second, is the camera system which sits underwater on the edge of the pool and is held in place by PVC frame that is help in place by a physical weight. These cameras are connected by wire to a WiFi router, which transmits their video feed over a local network.

At the center of the system, is the sensor consisting of the hydrophone, amplifier, analog interface, and laptop. The hydrophone sits about 1-2 feet under the surface of the water near the recording zone. The hydrophone is held in place by another PVC mount, which can either be connected to the wall of the pool or one of the lane lines. The cord from the hydrophone runs out of the pool and connects to the audio amplifier. The amp, analog interface, and laptop are not water resistant, so they are set up far away from the edge of the pool to avoid any splashing. The signal

from the amp is passed to the analog interface, which is connected to the laptop through its USB port. It is important that the laptop is not connected to power when the system is operating because the power supply will distort the analog signal.

The program on the computer acts as the centerpiece of the whole system. Ultrasonic sound signals are transmitted wirelessly through the water and are picked up by the sensing system. These signals are processed by the computer to determine if a swimmer is in the recording zone. If the beeper is close enough to the hydrophone, a strong signal is passed through the sensing system. If the signal is above a certain threshold, the system determines that there is a swimmers in the recording zone. The program then connects wirelessly to the camera and records a ten second clip. It is important that the laptop is on the same WiFi network as the camera, or else it will not be able to connect to the cameras. In simple terms, when the beeper gets near the sensor it beeps louder. If a beep is loud enough, the sensor will sense the beeper and start collecting footage from the camera. When testing, the beeper was held in the water by hand and moved along the edge of the pool.

In practice, the beeper is strapped to the side of the swimmers head using their goggle straps. The sensor can be placed any in the pool, as long as the cable from the hydrophone is long enough to get the amplifier on the side of the pool. Additionally, the camera can be placed anywhere because it connects to the laptop through a wireless network. Further, the closer the hydrophone is to the wall, to more noise there is in its data and the less accurate it is. The best results were obtained in a middle lane with the hydrophone about three feet from the wall. Additionally, the beeper must be underwater for the sensor to pick it up. In general, the sensor picks up the beeper when it they are about 5 yards apart from each other. While the hydrophone is omni-directional, the beeper is not. The signal is much stronger when one of its transducers is pointed towards the hydrophone.

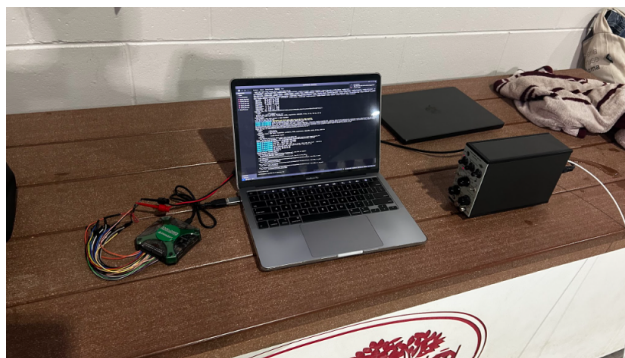


Figure 4: Sensing system components with the laptop displaying the processing program, the amplifier, and the Analog Discovery 2 connected

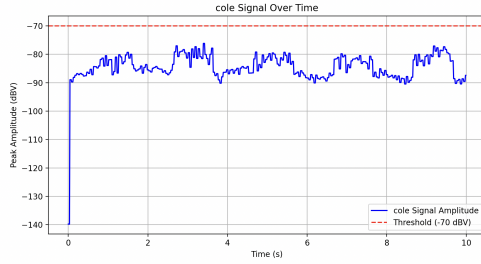
5 Results

To evaluate the success of the SwimForce system, a series of tests was conducted to verify detection accuracy, acoustic signal clarity, system responsiveness, and visual recording reliability. These tests were conducted under controlled pool conditions using the wearable ultrasonic transducer, hydrophone connected through the Analog Discovery 2, and the POE camera setup.

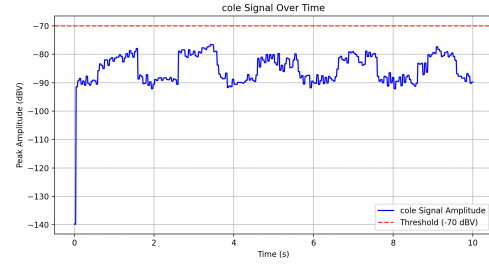
5.1 FFT Detection at Varying Distances

To assess the signal clarity and reliability of the detection software, FFT outputs were recorded and plotted while the ultrasonic transducer was positioned at varying distances from the hydrophone. These tests confirmed that the peak amplitude at the target frequency would surpass the threshold within the intended range.

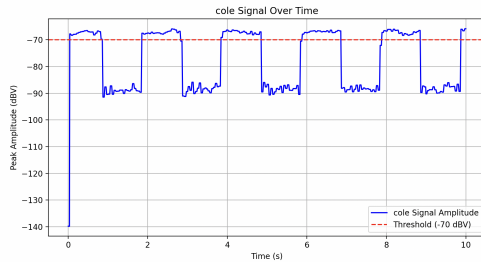
Each sub figure below shows the magnitude of the FFT result for a swimmer tag frequency at different distances:



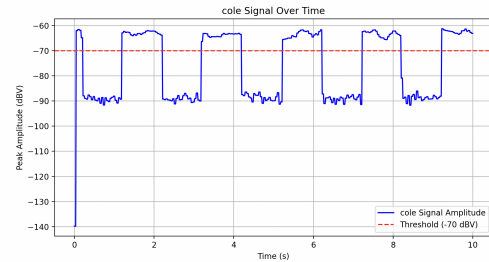
(a) FFT result at 5 yards from hydrophone



(b) FFT result at 4 yards from hydrophone



(c) FFT result at 3 yards from hydrophone



(d) FFT result at 1 yard from hydrophone

Figure 5: FFT spectra at varying swimmer distances from the hydrophone, showing signal amplitude near 40 kHz. Detection is consistent at all tested distances, with stronger peaks at closer range.

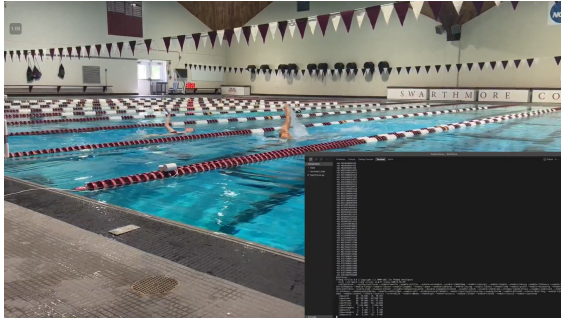
As shown in the figure above, as the beeper moves closer to the hydrophone, the clarity and amplitude of the detected ultrasonic signal improves significantly. This is evident in the FFT spectra as the peak in the sub figure (a) is present but buried in noise. As the distance closes, the peak becomes sharply defined and well above the noise floor. This improvement is due to the reduced acoustic attenuation and better coupling between the water and sensor at shorter distances. Additionally, when visualizing the raw time domain signal, the square wave pattern emitted by the transducer becomes more defined. This increased clarity not only confirms the effectiveness of the wearable beeper, but also validates the systems thresholding logic for real-time detection.

Although the images displayed in the figure indicated the need for the device to be much closer than desired, simple changes to the positioning of the hydrophone and lane used alleviated these issues. By moving the hydrophone away from any walls the system produced much cleaner, louder, and farther detections.

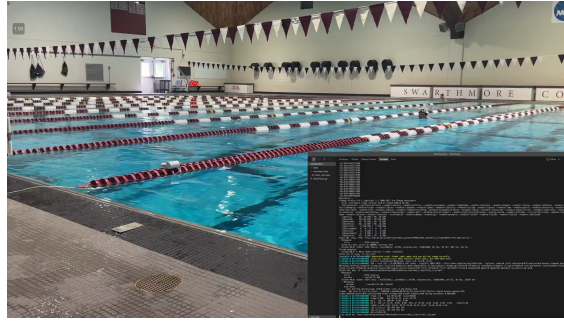
As shown, once the program is initiated, it begins to read live dBV signals. At that current time, the ultrasonic beeper was too far and so the resonant frequency that the hydrophone was picking up hovered -67 dBV. Once the ultrasonic signal was in range, the reading would increase to values between -30 and -59 dBV. Through extensive testing, the threshold was set at -57 dBV, ensuring the swimmer was close enough to be captured by the camera.

5.3 Synchronized Recording Triggered by Detection

14



(a) Confirmation of synchronized video recording triggered by ultrasonic detection.



(b) Confirmation that the clip was recorded.

The large text displayed in the program overlay on sub-figure (a) indicates that the .ffmpeg command was triggered and the camera began recording. As shown, the swimmer is directly under the flags (beginning of the cameras field of view) and the code has sent the command to the cameras to begin the video capture. Once the swimmer has swam into the wall, completed their turn, and pushed off the wall (which takes approximately 10 seconds) the camera stops recording and begins to initialize the next iteration as shown in sub-figure (b). Additionally, the folder shown on the left side of the program overlay shows the personalized clip being saved under the athletes name.

5.4 Underwater Visual Clarity and Spectrum Match

To confirm that visual footage is usable for analysis, and that the audio/video syncing is properly functioning, we captured the POE camera footage while also recording the frequency spectrum in WaveForms. The resulting frames show clear swimmer motion, and the corresponding spectrum output which confirms detection with a distinct 40 kHz peak.

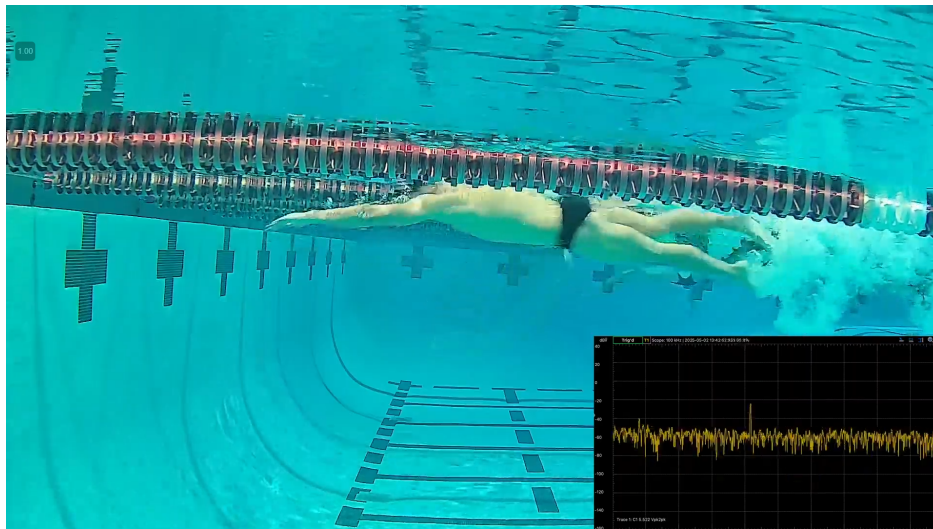


Figure 7: Underwater camera feed paired with WaveForms spectrum analysis showing a strong 40 kHz signal.

As shown in the figure above, the swimmer is in clear view of the camera. Though it is simply a screenshot from the actual recorded footage, it is apparent that the swimmers actions and flip turns can be analyzed in detail. Additionally, the WaveForms spectrum overlay shows a sample "beep" detected at that time by the hydrophone indicating clear signaling above the noise floor.

These results collectively demonstrate that the SwimForce system reliably detects swimmer specific ultrasonic signals, triggers the video capture, and provides clear underwater footage usable for stroke analysis. The FFT plots confirm successful frequency domain detection within the cameras field of view, the real time terminal output shows the system responding appropriately to the dBV threshold crossings, and the diluvial confirmation of the triggered recordings aligns with the swimmers presence in the clip. Combined with the underwater footage saved under the athletes name, these findings validate the functionality of SwimForce under realistic training conditions.

5.5 Limitations

There are several limitations to the current design that would need to be addressed for the system be practical in a real swim practice environment.

First, when the beeper is attached to the swimmers head, the hydrophone is only able to pick it up when the swimmers head is underwater. Sometimes when swimmers are breathing while swimming breaststroke and butterfly, their heads come out of the water and the beeper is unable to be picked up by the hydrophone.

Second, the current beeper operates at one frequency. In order to scale this system to multiple swimmers, different beepers would need to be able to operate at multiple different frequencies with enough resolution between frequencies. Further, the sensing system would need to be reconfigured to recognize multiple different frequencies.

Third, the sensing system is fragile and difficult to set up for someone without a technical understanding of the system. This makes the current set up impractical for commercial use. The sensor system would need to be consolidated into a plug and play device that is designed specifically for this application, rather than three different expensive and fragile components that have many different connections and points of failure.

Additionally, the tested beeper design is too large to be practical for a real practice environment. The beeper has no on/off switch, and needs to be opened with a screw driver for the battery to be plugged in or charged. For the beeper to be practical, it would need to be smaller, able to be charged easily and turned on/off easily.

6 Discussion & Conclusion

6.1 Impact

A technology like SwimForce has the potential to modernize swim practices from the club level all the way to the highest level of collegiate swimming. Many coaches are becoming more data driven in their approaches to swimming and a technology like SwimForce could enhance data collection on athlete performance and supplement other technologies that collect biometric data on athletes.

The ability to automatically track swimmers while they are in the pool opens up a door to automate many of the existing challenges of coaches and enable coaches and swimmers to find new coaching strategies. For example, collecting times (splits) in the middle of practice is a manual task that takes coaches attention away from other important swimmer to coach technical interactions. Coaches have to constantly check their watches and write down splits by hand as many different swimmers come into the wall at the same time. A technology like SwimForce could automate split collection, allowing coaches to spend more time analyzing swimmer technique and performance data. Coaches often only collect splits on specific sets as one coach can generally only collect splits on 2-3 lanes at a time and it takes multiple coaches to accurately collect splits on an entire practice group. The ability to collect splits on any set with ease with allows both coaches and swimmers to collect far more data in practice.

Additionally, collecting instantaneous velocity creates a new space of data that could be collected on swimmers in practice. Swimming is a sport of efficiency, where the best swimmers are talented at slowing down the least compared to others. Understanding how a swimmers velocity changes throughout a lap could reshape how swimmers and coaches think about turns, underwater dolphin kicks and stroke technique.

6.2 Next Steps

6.2.1 Velocity Calculation

Ideally, the next step in this project would involve calculating the velocity of the swimmer within the recording zone. This information would provide valuable performance metrics, such as speed approaching and leaving the pool wall on several different types of turns. This would allow coaches to give technical input on which turn to do (bucket turn, flip turn, open turn, or cross-over turn) while showing the speed changes for each.

Additionally, calculating velocity opens the door to even more in-depth analysis of their swimmers performances. For instance, it could quantify the speed of their dive by calculating their velocity as they enter the water, their acceleration or deceleration as they approach the wall for a turn, and their push-off velocity. These metrics are crucial for biomechanical analysis and could provide insights for improving training sessions.

The question then arises: “How does one calculate velocity using OpenCV?”

One approach to calculating the swimmer’s velocity from video footage involves using optical flow techniques, such as the Lucas-Kanade method. This method estimates the apparent motion of image pixels between two consecutive frames and uses the calculated meters to pixel ratio to find the actual velocity of a moving object.

Assuming pixel intensity remains constant and the motion is small, we begin with the brightness constancy assumption:

$$I(x, y, t) = I(x + u, y + v, t + 1)$$

where $I(x, y, t)$ is the intensity of the pixel at (x, y) at time t , and u and v are the displacement in the x and y directions (in pixels) between frames.

For small motions, this can be linearized using a Taylor series expansion:

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + I_x u + I_y v + I_t$$

where I_x and I_y are the image gradients (spatial derivatives), and I_t is the temporal derivative.

The Lucas-Kanade method uses a window of N pixels, assuming constant velocity within that window, and solves for u and v using least squares:

$$\begin{bmatrix} I_{x_1} & I_{y_1} \\ I_{x_2} & I_{y_2} \\ \vdots & \vdots \\ I_{x_N} & I_{y_N} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_{t_1} \\ I_{t_2} \\ \vdots \\ I_{t_N} \end{bmatrix}$$

$$\begin{bmatrix} u \\ v \end{bmatrix} = (A^T A)^{-1} A^T b$$

where A is the gradient matrix and b is the vector of temporal gradients (Lucas Kanade, 1981).

Once the pixel velocity (u, v) is obtained, it can be converted to real-world velocity (V_x, V_y) . First, the pixel velocity is calculated:

$$u = \frac{\Delta x}{\Delta t}, \quad v = \frac{\Delta y}{\Delta t}$$

where Δt is the time difference between frames (e.g., 1/30 for 30 fps).

Then, for a single camera setup, we need to determine the meters per pixel:

$$\text{meters per pixel} = \frac{\text{sensor width (m)} \times \text{object distance (m)}}{\text{focal length (m)} \times \text{image width (pixels)}}$$

Finally, the real-world velocity components are computed as follows:

$$V_x = u \times \text{meters per pixel} \div \Delta t$$

$$V_y = v \times \text{meters per pixel} \div \Delta t$$

If a double camera setup was used however, depth information (Z) can be estimated using disparity (d) between the cameras with a known baseline (B) and focal length (f):

$$Z = \frac{f \cdot B}{d}$$

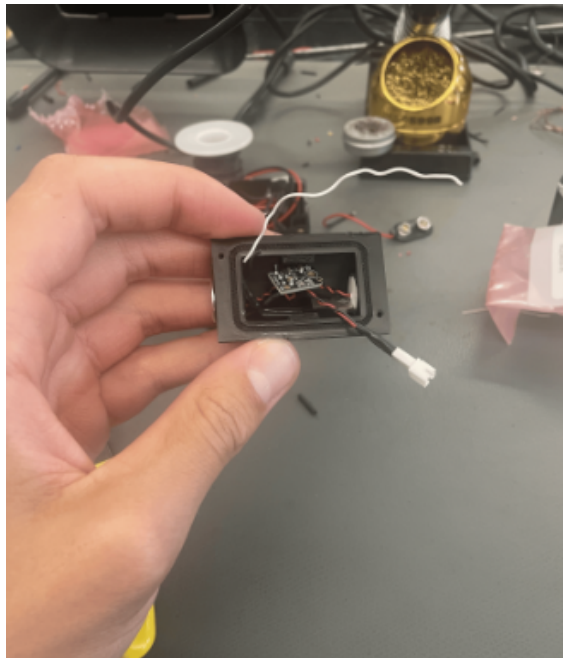
This depth information can then be used to convert pixel velocity to real-world velocity (Szeliski, 2022).

The question then becomes: “Why not calculate the velocity using OpenCV?”

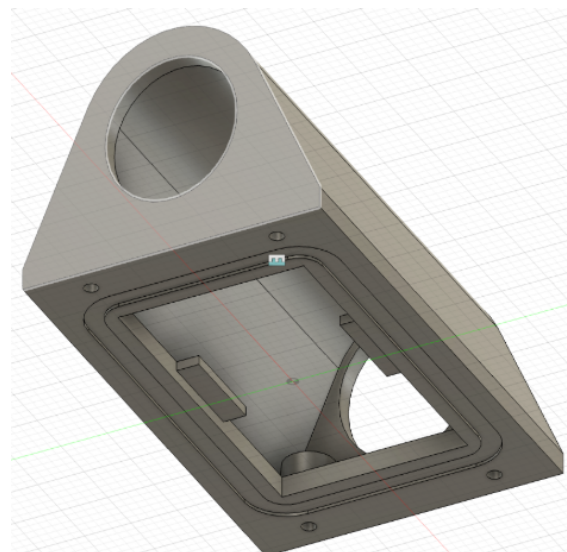
Implementing an accurate velocity calculation was deemed outside the scope of the current project due to several complexities and time constraints. Dynamic water surfaces can significantly disrupt object isolation, which is crucial for reliable optical flow analysis. A common initial step in motion analysis is to use background subtraction. The problem here is there are struggles with the constantly changing reflections and refractions caused by water movement. Furthermore, accurate camera calibration is essential for converting pixel measurements to real-world units. This process requires a fixed distance and angle, which can be challenging to maintain in a pool environment considering strong waves and water movement can shift the cameras swivel. Finally, the optical flow algorithms often require precise noise reduction in order to get reliable results, thus adding a layer of complexity that would have significantly extended the project timeline.

6.2.2 3D Printed design

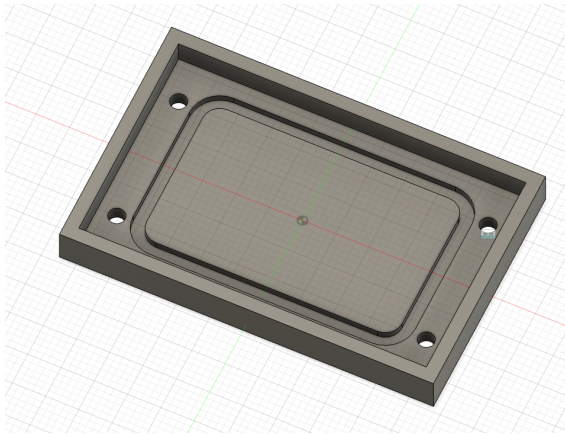
As a result, the focus turned to optimizing the systems design by creating a smaller wearable ultrasonic beeper. The case used in the tested design is an IP68 rated plastic box with holes cut into the sides. While this design worked for testing, the box is large and creates noticeable drag when worn in the pool. In order to make the design smaller, a custom case needed to be 3D printed that could house both transducers and all of the electronics while still being water proof. The case has two main components, a larger mailbox shaped box that holds all of the electronics with holes for the transducers on either side, and a lid that sealed off the bottom while still providing access to the battery inside the mailbox. The lid has a groove in it that is meant to house a rubber O-ring.



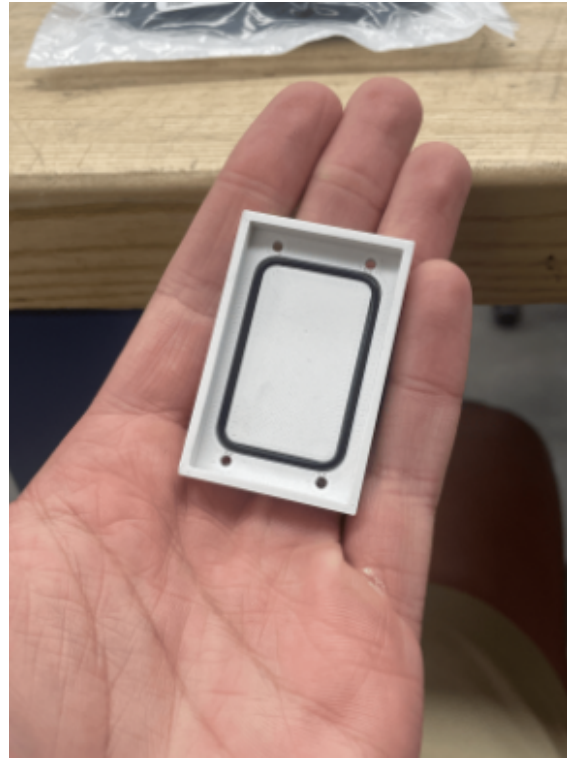
(a) 3D printed top with transducers glued in



(b) CAD: top "mailbox" design



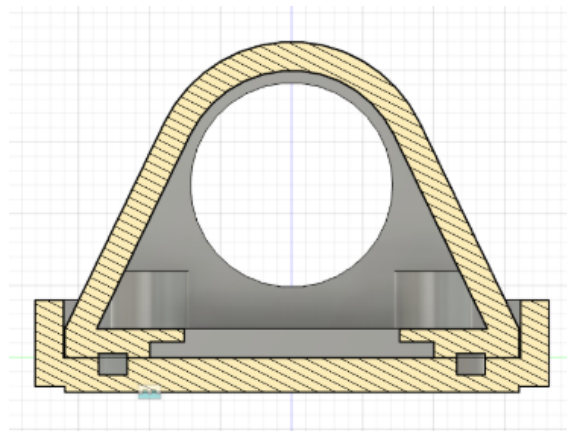
(c) CAD: lid design



(d) 3D printed lid with O-ring



(e) CAD: side view of full case



(f) Alternate side view

Figure 8: CAD and 3D printed views of the swimmer-worn ultrasonic housing, including lid design, seal integration, and side profiles.

APPENDIX A: Arduino Code

```
1 void setup() {  
2     pinMode(11, OUTPUT);  
3     TCCR2A = _BV(COM2A0) | _BV(COM2B1) | _BV(WGM20);  
4     TCCR2B = _BV(WGM22) | _BV(CS20);  
5     OCR2A = 52;  
6     OCR2B = 50;  
7 }  
8  
9 void loop() {  
10     pinMode(11, OUTPUT);  
11     delay(200);  
12  
13     pinMode(11, INPUT);  
14     delay(800);  
15 }
```

APPENDIX B: Camera Clipping Code

```
1 def rec_clip(durr, output_filename):
2     # Ensure the directory exists
3     os.makedirs(os.path.dirname(output_filename), exist_ok=True)
4
5     command = [
6         "ffmpeg",
7         "-rtsp_transport", "tcp",
8         "-y",
9         "-t", str(durr),
10        "-i", RTSP0, # First input stream
11        "-c:v", "libx264",
12        "-preset", "ultrafast",
13        "-crf", "23",
14        "-pix_fmt", "yuv420p",
15        output_filename
16    ]
17
18    print("Recording...")
19    try:
20        process = subprocess.Popen(command)
21        process.wait()
22        print(f"Clip saved as: {output_filename}")
23    except subprocess.CalledProcessError as e:
24        print("Error during ffmpeg recording:", e)
```

APPENDIX C: Audio Detection Code

```

1 SAMPLE_RATE = 250_000      # 250 kHz
2 BUFFER_SIZE = 16384        # Match WaveForms settings
3 PULSE_DURATION = 10        # Monitor for 10 seconds
4 ORDER = 4                  # Order of the bandpass filter
5 LOG_INTERVAL = 0.01        # Log data every 0.01 seconds
6 THRESHOLD_DBV = -95        # Threshold for logging timestamps
7
8 def read_data(hdwf):
9     data = (ctypes.c_double * BUFFER_SIZE)()
10    dwf.FDwfAnalogInStatus(hdwf, ctypes.c_bool(True), None)
11    dwf.FDwfAnalogInStatusData(hdwf, ctypes.c_int(0), data, ctypes.c_int(
        BUFFER_SIZE))
12    return np.array(data)
13
14 def butter_bandpass(lowcut, highcut, fs, order=5):
15     nyq = 0.5 * fs
16     low = lowcut / nyq
17     high = highcut / nyq
18     b, a = butter(order, [low, high], btype='band')
19     return b, a
20
21 def butter_bandpass_filter(data, lowcut, highcut, fs, order=5):
22     b, a = butter_bandpass(lowcut, highcut, fs, order=order)
23     y = lfilter(b, a, data)
24     return y
25
26 if hdwf:
27     configure_device(hdwf)
28     start_time = time.time()
29     last_log_time = start_time
30     collecting = True
31     print("Starting data acquisition...")
32
33     while True:
34         current_time = time.time()
35
36         # Toggle collecting every LOG_INTERVAL (creates a duty cycle to reduce
            CPU load)
37         if collecting and current_time - last_log_time >= LOG_INTERVAL:
38             last_log_time = current_time
39             collecting = False # Switch off for 1 interval
40         elif not collecting and current_time - last_log_time >= LOG_INTERVAL:
41             last_log_time = current_time
42             collecting = True # Switch back on
43
44         if collecting:
45             data = read_data(hdwf)
46
47             for name, (low, high) in PEOPLE.items():
48
49                 # Bandpass filter for this person's frequency band

```

```

50         filtered_data = butter_bandpass_filter(data, low, high,
51                                                 SAMPLE_RATE, order=ORDER)
52
53         # Apply windowing to reduce spectral leakage before FFT
54         window = np.blackman(len(filtered_data))
55         windowed = filtered_data * window
56         correction_factor = np.sum(window) / len(filtered_data)
57
58         # Compute FFT and convert to dBV
59         fft_result = np.fft.rfft(windowed) / (BUFFER_SIZE *
60                                             correction_factor)
61         magnitudes = 20 * np.log10(np.abs(fft_result) + 1e-12)
62
63         # Get the frequencies corresponding to FFT bins
64         freqs = np.fft.rfftfreq(len(filtered_data), 1 / SAMPLE_RATE)
65         target_idx = np.where((freqs >= low) & (freqs <= high))[0]
66
67         if len(target_idx) > 0:
68             peak_dbv = np.max(magnitudes[target_idx])
69             timestamp = current_time - start_time
70             print(peak_dbv)
71             # Trigger recording if signal exceeds threshold and not
72             already recording *THIS IS THE CRITICAL PART WE NEED
73             TO TEST IN DEPTH*
74             if peak_dbv < THRESHOLD_DBV and not recording_active[name
75                 ]:
76
77                 # Check for new threshold crossing
78                 swim_ins[name] += 1
79                 last_crossed[name] = True
80                 recording_active[name] = True
81
82                 output_filename = f"{name}_clip.mp4"
83                 rec_clip(PULSE_DURATION, output_filename)
84
85                 time.sleep(PULSE_DURATION)
86                 recording_active[name] = False
87             else:
88                 last_crossed[name] = False
89
90         close_device(hdwf)
91         print("Data acquisition complete.")
92     else:
93         print("Failed to open the device.")

```

WORKS CITED

Brüel Kjør. (1989). Charge Amplifier Type 2635: Product Data (BP 0099). <https://www.bksv.com/-/media/literature/Product-Data/bp0099.ashx>

Adafruit Industries. (n.d.). Product 5644: DC/DC Step-Up Power Module [Product page]. Digi-Key Electronics. <https://www.digikey.com/en/products/detail/adafruit\protect\penalty-\@M-industries-llc/5644/24639126>

Arduino. (n.d.). Arduino Pro Mini 3.3V Getting Started Guide [PDF]. Digi-Key Electronics. https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/492/3.3V_GettingStarted_Web.pdf

Computer Vision Textbook (General Optical Flow, Camera Geometry): Szeliski, R. (2022). Computer vision: Algorithms and applications (2nd ed.). Springer. <https://szeliski.org/Book/>

Digilent. (n.d.). Analog Discovery 2 Reference Manual. Retrieved May 9, 2025, from <https://digilent.com/reference/test-and-measurement/analog-discovery-2/reference\protect\penalty-\@M-manual>

DTP. (n.d.). DTP401525 110mAh Lithium Polymer Battery Datasheet [PDF]. SparkFun Electronics. <https://cdn.sparkfun.com/datasheets/Prototyping/spe-00-DTP401525\protect\penalty-\@M-110mah-en-1.0ver.pdf>

Hottinger Brüel Kjør A/S. (2021, November). Hydrophone Types 8103, 8104, 8105 and 8106 (Product data: BP 0317). <https://www.bksv.com/media/doc/bp0317.pdf>

Lucas-Kanade Method (Original Paper): Lucas, B. D., Kanade, T. (1981). An iterative image registration technique with an application to stereo vision. In Proceedings of the 7th International Joint Conference on Artificial Intelligence (pp. 674–679).

NOAA Ocean Exploration. (2002, April 24). Understanding ocean acoustics. <https://oceanexplorer.noaa.gov/explorations/sound01/background/acoustics/acoustics.html>

Onsemi. (2017, November). TIP120 / TIP121 / TIP122 NPN Epitaxial Darlington Transistor (Rev. 2) [Data sheet]. <https://www.onsemi.com/download/data-sheet/pdf/tip122-d.pdf>

Raculety. (2020). 2 Pack Project Box IP65 Waterproof Junction Box ABS Plastic Black Electrical Boxes DIY Electronic Project Case Power Enclosure 2.48"x2.28"x1.38" (63x58x35mm) [Product page]. Amazon. <https://www.amazon.com/Junction-Raculety-Electrical--Power/dp/B08ZDG73XB>

SAFR Support. (2021, August 4). Record RTSP streams - ffmpeg. <https://support.safr.>

com/support/solutions/articles/69000145615-record-rtsp-streams

Same Sky. (2024, September 12). CUSA-TR65-065-2200-W68 Datasheet: Ultrasonic Sensor. <https://www.sameskydevices.com/product/resource/cusa-tr65-065-2200\protect\penalty-\@M-w68.pdf>

Same Sky. (2024, September 12). CUSA-TR80-065-2000-TH68 Datasheet: Ultrasonic Sensor. <https://www.sameskydevices.com/product/resource/cusa-tr80-065-2000-th68.pdf>

Same Sky. (2024, September 12). CUSA-TR801-065-2000-TH68 Datasheet: Ultrasonic Sensor. <https://www.sameskydevices.com/product/resource\protect\penalty-\@M/cusa-tr801-065\protect\penalty-\@M-2000-th68.pdf>

Sideline Scout. (n.d.). Sideline Scout underwater camera - 1080P [Bundle-Cameras]. Retrieved May 9, 2025, from <https://sidelinescout.com/product/uncategorized/bundle\protect\penalty-\@M-cameras/sideline-scout-underwater-camera-1080p/>